

6. QR factorization

- QR factorization
- QR via Gram-Schmidt
- modified Gram-Schmidt
- pivoted QR factorization
- Householder algorithm

QR factorization

any $A \in \mathbb{R}^{m \times n}$ with linearly independent columns admits the factorization:

$$A = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix} \begin{bmatrix} R_{11} & R_{12} & \cdots & R_{1n} \\ 0 & R_{22} & \cdots & R_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & R_{nn} \end{bmatrix} = QR$$

- Q is $m \times n$ with orthonormal columns ($Q^T Q = I$)
- R is $n \times n$, upper triangular, with nonzero diagonal elements (invertible)
- most definitions assume $R_{ii} > 0$ making Q and R unique:
 - if $R_{ii} < 0$, we can switch the signs of $R_{ii}, \dots, R_{in}$ and vector q_i
- called *QR factorization* (also known as *reduced* or *thin* QR factorization)
- several algorithms (in MATLAB: $[Q, R] = \text{qr}(A, 0)$), including Gram-Schmidt

Pseudo-inverse via QR factorization

pseudo-inverse of A with linearly independent columns with $A = QR$ is

$$\begin{aligned}A^{\dagger} &= (A^T A)^{-1} A^T \\&= ((QR)^T (QR))^{-1} (QR)^T \\&= (R^T Q^T Q R)^{-1} R^T Q^T \\&= (R^T R)^{-1} R^T Q^T \quad (Q^T Q = I) \\&= R^{-1} R^{-T} R^T Q^T \quad (R \text{ is nonsingular}) \\&= R^{-1} Q^T\end{aligned}$$

- for square nonsingular A this is the inverse: $A^{-1} = (QR)^{-1} = R^{-1} Q^T$
- pseudo-inverse of A with linearly independent rows with $A^T = \hat{Q} \hat{R}$ is

$$A^{\dagger} = A^T (A A^T)^{-1} = \hat{Q} \hat{R}^{-T}$$

Range and QR factorization

suppose A has linearly independent columns with QR factorization $A = QR$

- Q has the same range as A :

$$\begin{aligned}y \in \text{range}(A) &\iff y = Ax \text{ for some } x \\&\iff y = QRx \text{ for some } x \\&\iff y = Qz \text{ for some } z \\&\iff y \in \text{range}(Q)\end{aligned}$$

- columns of Q are *orthonormal basis* for $\text{range}(A)$:
they are linearly independent and $\text{span}(q_1, \dots, q_n) = \text{range}(A)$

Full QR factorization

the QR factorization of $A \in \mathbb{R}^{m \times n}$ with lin. indep. columns is usually defined as

$$A = \begin{bmatrix} Q & Q_0 \end{bmatrix} \begin{bmatrix} R \\ 0 \end{bmatrix}$$

- $A = QR$ with same Q, R as before
- $\begin{bmatrix} Q & Q_0 \end{bmatrix}$ is $m \times m$ and orthogonal
- Q_0 has size $m \times (m - n)$ and zero block has size $(m - n) \times n$
- several algorithms, including Householder; in MATLAB: $[Q, R] = \text{qr}(A)$
- called *full QR factorization* or *QR decomposition*

Outline

- QR factorization
- **QR via Gram-Schmidt**
- modified Gram-Schmidt
- pivoted QR factorization
- Householder algorithm

Matrix form of Gram-Schmidt

let A be an $m \times n$ matrix with linearly independent columns

- running Gram-Schmidt on A produces orthonormal vectors $q_1, \dots, q_n$
- we know from Gram-Schmidt algorithm that

$$\begin{aligned}a_k &= (q_1^T a_k)q_1 + \dots + (q_{k-1}^T a_k)q_{k-1} + \|\tilde{q}_k\|q_k \\ &= R_{1k}q_1 + \dots + R_{k-1,k}q_{k-1} + R_{kk}q_k\end{aligned}$$

where $R_{ij} = q_i^T a_j$ and $R_{ii} = \|\tilde{q}_i\| > 0$

- expressing this for each $k = 1, \dots, n$:

$$a_1 = R_{11}q_1$$

$$a_2 = R_{12}q_1 + R_{22}q_2$$

$$\vdots$$

$$a_n = R_{1n}q_1 + \dots + R_{nn}q_n$$

$$A = [q_1 \quad \dots \quad q_n] \begin{bmatrix} R_{11} & R_{12} & \dots & R_{1n} \\ 0 & R_{22} & \dots & R_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & R_{nn} \end{bmatrix}$$

QR factorization via Gram-Schmidt

given: $m \times n$ matrix A with linearly independent columns $a_1, \dots, a_n$

set $q_1 = a_1 / \|a_1\|$ and $R_{11} = \|a_1\|$

for $k = 2, \dots, n$

1. $\tilde{q}_k = a_k$

2. **for** $j = 1, \dots, k - 1$

$$R_{jk} = q_j^T a_k$$

$$\tilde{q}_k = \tilde{q}_k - R_{jk} q_j$$

3. **set**

$$R_{kk} = \|\tilde{q}_k\|$$

$$q_k = \tilde{q}_k / R_{kk}$$

- Q and R is generated column by column
- **complexity:** $2mn^2$ flops

Gram-Schmidt algorithm after k steps

after k steps we have a partial QR factorization

$$\begin{bmatrix} a_1 & a_2 & \cdots & a_k \end{bmatrix} = \begin{bmatrix} q_1 & q_2 & \cdots & q_k \end{bmatrix} \begin{bmatrix} R_{11} & R_{12} & \cdots & R_{1k} \\ 0 & R_{22} & \cdots & R_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & R_{kk} \end{bmatrix}$$

- this is the QR factorization for the first k columns of A
- columns $q_1, \dots, q_k$ are orthonormal basis for $\text{span}(a_1, \dots, a_k)$
- diagonal elements $R_{11}, R_{22}, \dots, R_{kk}$ are positive
- in step k of the algorithm we compute column k of Q and R :

$$q_k, R_{1k}, \dots, R_{kk}$$

Example

$$A = [a_1 \ a_2 \ a_3] = \begin{bmatrix} -1 & -1 & 1 \\ 1 & 3 & 3 \\ -1 & -1 & 5 \\ 1 & 3 & 7 \end{bmatrix}$$

- $k = 1$ (first column of Q and R)

$$q_1 = a_1 / \|a_1\| = (-1/2, 1/2, -1/2, 1/2), \quad R_{11} = \|a_1\| = 2$$

- $k = 2$ (second column of Q and R)

$$R_{12} = q_1^T a_2 = 4, \quad \tilde{q}_2 = a_2 - R_{12}q_1 = (1, 1, 1, 1)$$

normalizing, we get

$$R_{22} = \|\tilde{q}_2\| = 2, \quad q_2 = \tilde{q}_2 / R_{22} = (1/2, 1/2, 1/2, 1/2)$$

Example

- $k = 3$ (third column of Q and R)

$$R_{13} = q_1^T a_3 = 2, \quad R_{23} = q_2^T a_3 = 8$$

$$\tilde{q}_3 = a_3 - R_{13}q_1 - R_{23}q_2 = (-2, -2, 2, 2)$$

normalizing, we get

$$R_{33} = \|\tilde{q}_3\| = 4, \quad q_3 = \tilde{q}_3/R_{33} = (-1/2, -1/2, 1/2, 1/2)$$

Final factorization

$$\begin{aligned} \begin{bmatrix} -1 & -1 & 1 \\ 1 & 3 & 3 \\ -1 & -1 & 5 \\ 1 & 3 & 7 \end{bmatrix} &= [q_1 \quad q_2 \quad q_3] \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ 0 & R_{22} & R_{23} \\ 0 & 0 & R_{33} \end{bmatrix} \\ &= \begin{bmatrix} -1/2 & 1/2 & -1/2 \\ 1/2 & 1/2 & -1/2 \\ -1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & 1/2 \end{bmatrix} \begin{bmatrix} 2 & 4 & 2 \\ 0 & 2 & 8 \\ 0 & 0 & 4 \end{bmatrix} \end{aligned}$$

Numerical instability of G-S

consider the following MATLAB implementation of the G-S algorithm

```
[m, n] = size(A);  
Q = zeros(m,n);  
R = zeros(n,n);  
for k = 1:n  
    R(1:k-1,k) = Q(:,1:k-1)' * A(:,k);  
    qtilde = A(:,k) - Q(:,1:k-1) * R(1:k-1,k);  
    R(k,k) = norm(qtilde);  
    Q(:,k) = qtilde / R(k,k);  
end;
```

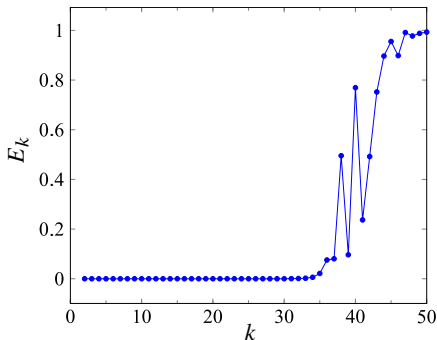
- we apply this to a square matrix A of size $m = n = 50$
- A is constructed as $A = USV$ with U, V orthogonal, S diagonal with

$$S_{ii} = 10^{-10(i-1)/(n-1)}, \quad i = 1, \dots, n$$

Numerical instability of G-S

plot shows deviation from orthogonality between q_k and previous columns

$$E_k = \max_{1 \leq i < k} |q_i^T q_k|, \quad k = 2, \dots, n$$



loss of orthogonality is due to rounding error

Outline

- QR factorization
- QR via Gram-Schmidt
- **modified Gram-Schmidt**
- pivoted QR factorization
- Householder algorithm

Modified Gram-Schmidt algorithm

a variation of the classical Gram-Schmidt algorithm for QR factorization

$$\begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix} \begin{bmatrix} R_{11} & R_{12} & \cdots & R_{1n} \\ 0 & R_{22} & \cdots & R_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & R_{nn} \end{bmatrix}$$

of a matrix with linearly independent columns

- mathematically equivalent to classical G-S
- has better numerical properties than classical Gram-Schmidt algorithm
- computes Q column by column, R row by row

Gram-Schmidt orthogonalization

recall G-S: for $k = 1, \dots, n$

$$\tilde{q}_k = a_k - \sum_{j=1}^{k-1} (q_j^T a_k) q_j \quad (R_{jk} = q_j^T a_k)$$

$$R_{kk} = \|\tilde{q}_k\|, \quad q_k = \frac{\tilde{q}_k}{R_{kk}}$$

Projection interpretation

- recall the Gram-Schmidt computes the projection of a_k onto

$$\text{span}\{a_1, \dots, a_{k-1}\} = \text{span}\{q_1, \dots, q_{k-1}\}$$

and subtracts it from a_k (for $k \geq 2$):

$$\tilde{q}_k = (I - \sum_{j=1}^{k-1} q_j q_j^T) a_k = (I - [q_1 \cdots q_{k-1}][q_1 \cdots q_{k-1}]^T) a_k$$

- this is the orthogonal projector onto $\text{span}\{a_1, \dots, a_{k-1}\}^\perp$

Modified Gram-Schmidt: idea

- the orthogonal projector can be written as a product of elementary projections

$$I - \sum_{j=1}^{k-1} q_j q_j^T = (I - q_{k-1} q_{k-1}^T) \cdots (I - q_1 q_1^T)$$

- thus the residual vector for column i can be written as

$$\tilde{q}_i = (I - q_{i-1} q_{i-1}^T) \cdots (I - q_1 q_1^T) a_i, \quad i \geq 2$$

- after computing $q_k = \frac{\tilde{q}_k}{\|\tilde{q}_k\|}$, we orthogonalize all remaining columns $j > k$:

$$R_{kj} = q_k^T \tilde{q}_j \quad (\text{gives } R_{kj} = q_k^T a_j \text{ due to } q_k \perp q_i \text{ for } i < k)$$

$$\tilde{q}_j := (I - q_k q_k^T) \tilde{q}_j = \tilde{q}_j - R_{kj} q_k, \quad j = k+1, \dots, n$$

we start with $\tilde{q}_j = a_j$

- this can be formulated into the modified G-S algorithm next

Modified Gram-Schmidt algorithm

given: $m \times n$ matrix A with linearly independent columns $a_1, \dots, a_n$

set $\tilde{Q} = A$

for $k = 1, \dots, n$

1. set

$$R_{kk} = \|\tilde{q}_k\|$$

$$q_k = \tilde{q}_k / R_{kk}$$

2. **for** $j = k + 1, \dots, n$

$$R_{kj} = q_k^T \tilde{q}_j$$

$$\tilde{q}_j = \tilde{q}_j - R_{kj} q_k$$

- R is generated *row by row* (at step k , we compute the k th row $R_{k,k:n}$)
- **complexity:** $2mn^2$ flops

Modified Gram-Schmidt in matrix-vector form

given: $m \times n$ matrix A with linearly independent columns $a_1, \dots, a_n$

set $\tilde{Q}_0 = A$

for $k = 1, \dots, n$

1. compute $R_{kk} = \|\tilde{q}_k\|$ and $q_k = (1/R_{kk})\tilde{q}_k$ where $\tilde{q}_k$ is the first column of $\tilde{Q}_{k-1}$
2. compute

$$[R_{k,k+1} \cdots R_{kn}] = q_k^T B, \quad \tilde{Q}_k = B - q_k [R_{k,k+1} \cdots R_{kn}]$$

where B is $\tilde{Q}_{k-1}$ with the first column removed

MATLAB implementation ($Q(:, k:n)$ used to store $\tilde{Q}_{k-1}$)

```
Q = A; R = zeros(n,n);  
for k = 1:n  
    R(k,k) = norm(Q(:,k));  
    Q(:,k) = Q(:,k) / R(k,k);  
    R(k,k+1:n) = Q(:,k)' * Q(:,k+1:n);  
    Q(:,k+1:n) = Q(:,k+1:n) - Q(:,k) * R(k,k+1:n);  
end;
```

Modified Gram-Schmidt algorithm after k steps

after k steps ($k = 1, \dots, n$), the algorithm has computed a partial QR factorization

$$A = [a_1 \cdots a_k \mid a_{k+1} \cdots a_n]$$
$$= [q_1 \cdots q_k \mid \tilde{Q}_k] \left[\begin{array}{ccc|ccc} R_{11} & \cdots & R_{1k} & R_{1,k+1} & \cdots & R_{1n} \\ \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & \cdots & R_{kk} & R_{k,k+1} & \cdots & R_{kn} \\ \hline & & 0 & & I & \end{array} \right]$$

- $q_1, \dots, q_k$ are orthonormal vectors; $R_{11}, \dots, R_{kk}$ are positive
- columns of $\tilde{Q}_k$ are projection of $a_{k+1}, \dots, a_n$ on $\text{span}(q_1, \dots, q_k)^\perp$
- the factorization starts with $\tilde{Q}_0 = A$ and is complete when $k = n$
- in step k , we compute

$$q_k, R_{kk}, R_{k,k+1}, \dots, R_{kn}, \tilde{Q}_k$$

Example

$$\begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} = \begin{bmatrix} -1 & -1 & 1 \\ 1 & 3 & 3 \\ -1 & -1 & 5 \\ 1 & 3 & 7 \end{bmatrix}$$

Step 1: first column of Q , first row of R

$$\begin{aligned} \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} &= \left[\begin{array}{c|cc} -1/2 & 1 & 2 \\ 1/2 & 1 & 2 \\ -1/2 & 1 & 6 \\ 1/2 & 1 & 6 \end{array} \right] \left[\begin{array}{c|cc} 2 & 4 & 2 \\ \hline 0 & 1 & 0 \\ 0 & 0 & 1 \end{array} \right] \\ &= \left[\begin{array}{c|c} q_1 & \tilde{Q}_2 \end{array} \right] \left[\begin{array}{c|c} R_{11} & R_{1,2:3} \\ \hline 0 & I \end{array} \right] \end{aligned}$$

Example

Step 2: second column of Q , second row of R

$$\begin{aligned} \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} &= \begin{bmatrix} -1/2 & 1/2 & -2 \\ 1/2 & 1/2 & -2 \\ -1/2 & 1/2 & 2 \\ 1/2 & 1/2 & 2 \end{bmatrix} \left[\begin{array}{cc|c} 2 & 4 & 2 \\ 0 & 2 & 8 \\ \hline 0 & 0 & 1 \end{array} \right] \\ &= \begin{bmatrix} q_1 & q_2 & \tilde{Q}_3 \end{bmatrix} \left[\begin{array}{cc|c} R_{11} & R_{12} & R_{13} \\ 0 & R_{22} & R_{23} \\ \hline 0 & 0 & 1 \end{array} \right] \end{aligned}$$

Step 3: third column of Q , third row of R

$$\begin{aligned} \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} &= \begin{bmatrix} -1/2 & 1/2 & -1/2 \\ 1/2 & 1/2 & -1/2 \\ -1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & 1/2 \end{bmatrix} \left[\begin{array}{ccc} 2 & 4 & 2 \\ 0 & 2 & 8 \\ 0 & 0 & 4 \end{array} \right] \\ &= \begin{bmatrix} q_1 & q_2 & q_3 \end{bmatrix} \left[\begin{array}{ccc} R_{11} & R_{12} & R_{13} \\ 0 & R_{22} & R_{23} \\ 0 & 0 & R_{33} \end{array} \right] \end{aligned}$$

Outline

- QR factorization
- QR via Gram-Schmidt
- modified Gram-Schmidt
- **pivoted QR factorization**
- Householder algorithm

QR factorization with column pivoting

A is an $m \times n$ matrix (may be wide or have linearly dependent columns)

QR factorization with column pivoting (column reordering)

$$A = QRP^T$$

- Q is $m \times r$ with orthonormal columns
- R is $r \times n$, leading $r \times r$ submatrix is upper triangular with nonzero diagonal:

$$R = [\begin{array}{c|c} R_1 & R_2 \end{array}] = \left[\begin{array}{cccc|ccc} R_{11} & R_{12} & \cdots & R_{1r} & R_{1,r+1} & \cdots & R_{1n} \\ 0 & R_{22} & \cdots & R_{2r} & R_{2,r+1} & \cdots & R_{2n} \\ \vdots & \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & R_{rr} & R_{r,r+1} & \cdots & R_{rn} \end{array} \right]$$

- can be chosen so that $R_{11} \geq R_{22} \geq \cdots \geq R_{rr} > 0$
- P is an $n \times n$ permutation matrix
- r is the rank of A : gives full rank factorization $A = BC$ with $B = Q$, $C = RP^T$

Interpretation

- columns of $AP = QR$ are the columns of A in a different order
- the columns are divided in two groups:

$$AP = [\hat{A}_1 \ \hat{A}_2] = Q[R_1 \ R_2], \quad \hat{A}_1 \text{ is } m \times r, \ R_1 \text{ is } r \times r$$

- $\hat{A}_1 = QR_1$ is $m \times r$ with linearly independent columns:

$$\hat{A}_1 x = QR_1 x = 0 \implies R_1^{-1} Q^T \hat{A}_1 x = x = 0$$

- $\hat{A}_2 = QR_2$ is $m \times (n - r)$: columns are linear combinations of columns of $\hat{A}_1$

$$\hat{A}_2 = QR_2 = \hat{A}_1 R_1^{-1} R_2$$

the factorization provides two useful bases for $\text{range}(A)$

- columns of Q are an orthonormal basis
- columns of $\hat{A}_1$ are a basis selected from the columns of A

Basis for nullspace

Basis for nullspace: a basis for the nullspace of $A \in \mathbb{R}^{m \times n}$ is given by columns of

$$P \begin{bmatrix} -R_1^{-1}R_2 \\ I \end{bmatrix} \in \mathbb{R}^{n \times (n-r)}$$

where P, R_1, R_2 are the matrices in the pivoted QR factorization

$$AP = Q \begin{bmatrix} R_1 & R_2 \end{bmatrix}$$

- P is a $n \times n$ permutation matrix
- Q is $m \times r$ with orthonormal columns, where $r = \text{rank}(A)$
- R_1 is $r \times r$ upper triangular and nonsingular, R_2 is $r \times (n - r)$

Dimension of nullspace: the above result implies that

$$\dim(\text{null}(A)) = n - \text{rank}(A)$$

$\dim(\text{null}(A))$ is known as the *nullity* of the matrix

Proof

- x is in the nullspace of A if and only if $y = P^T x$ is in the nullspace of AP
- $y = (y_1, y_2)$ is in the nullspace of AP if and only if

$$\begin{aligned} APy = 0 &\iff Q \begin{bmatrix} R_1 & R_2 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = 0 \\ &\iff \begin{bmatrix} R_1 & R_2 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = 0 \quad (Q \text{ has orthonormal columns}) \\ &\iff \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -R_1^{-1}R_2 \\ I \end{bmatrix} y_2 \quad (R_1 \text{ nonsingular}) \end{aligned}$$

- therefore, x is in the nullspace of A if and only if it is in the range of

$$P \begin{bmatrix} -R_1^{-1}R_2 \\ I \end{bmatrix}$$

- the columns of this matrix are linearly independent, so they are a basis for

$$\text{range} \left(P \begin{bmatrix} -R_1^{-1}R_2 \\ I \end{bmatrix} \right) = \text{null}(A)$$

Modified Gram-Schmidt with column pivoting

with minor changes the modified GS algorithm computes the pivoted factorization

$$AP = \begin{bmatrix} q_1 & q_2 & \cdots & q_r \end{bmatrix} \left[\begin{array}{cccc|ccc} R_{11} & R_{12} & \cdots & R_{1r} & R_{1,r+1} & \cdots & R_{1n} \\ 0 & R_{22} & \cdots & R_{2r} & R_{2,r+1} & \cdots & R_{2n} \\ \vdots & \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & R_{rr} & R_{r,r+1} & \cdots & R_{rn} \end{array} \right]$$

- partial factorization after k steps

$$AP_k = \begin{bmatrix} q_1 & \cdots & q_k & | & \tilde{Q}_k \end{bmatrix} \left[\begin{array}{ccc|ccc} R_{11} & \cdots & R_{1k} & R_{1,k+1} & \cdots & R_{1n} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & R_{kk} & R_{k,k+1} & \cdots & R_{kn} \\ \hline & & 0 & & & I \end{array} \right]$$

- if $\tilde{Q}_k = 0$, the factorization is complete ($r = k, P = P_k$)
- algorithm starts with $P_0 = I$ and $\tilde{Q}_0 = A$
- before step k , we reorder columns of $\tilde{Q}_{k-1}$ to place its largest column first
- this requires reordering columns $k, \dots, n$ of R , and modifying P_{k-1}

Example

$$A = \begin{bmatrix} a_2 & a_1 & a_3 & a_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & -1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & -1 & -1 \end{bmatrix}$$

Step 1

- a_2 and a_4 have the largest norms; we move a_2 to the first position
- find first column of Q , first row of R

$$\begin{aligned} \begin{bmatrix} a_2 & a_1 & a_3 & a_4 \end{bmatrix} &= \left[\begin{array}{c|ccc} 1/2 & 1/2 & 0 & 1 \\ 1/2 & -1/2 & 1 & -1 \\ 1/2 & 1/2 & 0 & 1 \\ 1/2 & -1/2 & -1 & -1 \end{array} \right] \left[\begin{array}{c|ccc} 2 & 1 & 0 & 0 \\ \hline 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right] \\ &= \begin{bmatrix} q_1 & \tilde{Q}_1 \end{bmatrix} \left[\begin{array}{c|ccc} R_{11} & R_{1,2:4} \\ \hline 0 & I \end{array} \right] \end{aligned}$$

Example

Step 2

- move column 3 of $\tilde{Q}_1$ to first position in $\tilde{Q}_1$

$$\begin{bmatrix} a_2 & a_1 & a_3 & a_4 \end{bmatrix} = \left[\begin{array}{c|ccc} 1/2 & 1 & 1/2 & 0 \\ 1/2 & -1 & -1/2 & 1 \\ 1/2 & 1 & 1/2 & 0 \\ 1/2 & -1 & -1/2 & -1 \end{array} \right] \left[\begin{array}{c|ccc} 2 & 0 & 1 & 0 \\ \hline 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right]$$

- find second column of Q , second row of R

$$\begin{aligned} \begin{bmatrix} a_2 & a_1 & a_3 & a_4 \end{bmatrix} &= \left[\begin{array}{cc|cc} 1/2 & 1/2 & 0 & 0 \\ 1/2 & -1/2 & 0 & 1 \\ 1/2 & 1/2 & 0 & 0 \\ 1/2 & -1/2 & 0 & -1 \end{array} \right] \left[\begin{array}{cc|cc} 2 & 0 & 1 & 0 \\ 0 & 2 & 1 & 0 \\ \hline 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right] \\ &= \begin{bmatrix} q_1 & q_2 & | & \tilde{Q}_2 \end{bmatrix} \left[\begin{array}{cc|c} R_{11} & R_{12} & R_{1,3:4} \\ 0 & R_{22} & R_{2,3:4} \\ \hline 0 & 0 & I \end{array} \right] \end{aligned}$$

Example

Step 3

- move column 2 of $\tilde{Q}_2$ to first position in $\tilde{Q}_2$

$$\begin{bmatrix} a_2 & a_1 & a_3 & a_4 \end{bmatrix} = \left[\begin{array}{cc|cc} 1/2 & 1/2 & 0 & 0 \\ 1/2 & -1/2 & 1 & 0 \\ 1/2 & 1/2 & 0 & 0 \\ 1/2 & -1/2 & -1 & 0 \end{array} \right] \left[\begin{array}{cc|cc} 2 & 0 & 0 & 1 \\ 0 & 2 & 0 & 1 \\ \hline 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right]$$

- find third column of Q , third row of R

$$\begin{aligned} \begin{bmatrix} a_2 & a_1 & a_3 & a_4 \end{bmatrix} &= \left[\begin{array}{ccc|c} 1/2 & 1/2 & 0 & 0 \\ 1/2 & -1/2 & 1/\sqrt{2} & 0 \\ 1/2 & 1/2 & 0 & 0 \\ 1/2 & -1/2 & -1/\sqrt{2} & 0 \end{array} \right] \left[\begin{array}{ccc|c} 2 & 0 & 0 & 1 \\ 0 & 2 & 0 & 1 \\ 0 & 0 & \sqrt{2} & 0 \\ \hline 0 & 0 & 0 & 1 \end{array} \right] \\ &= \left[\begin{array}{ccc|c} q_1 & q_2 & q_3 & \tilde{Q}_3 \end{array} \right] \left[\begin{array}{ccc|c} R_{11} & R_{12} & R_{13} & R_{14} \\ 0 & R_{22} & R_{23} & R_{24} \\ 0 & 0 & R_{33} & R_{34} \\ \hline 0 & 0 & 0 & 1 \end{array} \right] \end{aligned}$$

Example

Final result: since $\tilde{Q}_3$ is zero, the algorithm terminates with the factorization

$$\begin{bmatrix} a_2 & a_1 & a_3 & a_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & -1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & -1 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & -1/2 & 1/\sqrt{2} \\ 1/2 & 1/2 & 0 \\ 1/2 & -1/2 & -1/\sqrt{2} \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 & 1 \\ 0 & 2 & 0 & 1 \\ 0 & 0 & \sqrt{2} & 0 \end{bmatrix}$$

Full pivoted QR factorization

any $A \in \mathbb{R}^{m \times n}$ admits the full pivoted QR factorization:

$$A = [Q \quad Q_0] \begin{bmatrix} R_1 & R_2 \\ 0 & 0 \end{bmatrix} P^T$$

- $Q, R = [R_1 \mid R_2]$ ($R_2 \in \mathbb{R}^{r \times (n-r)}$), and permutation P , are same as page 6.21
- $[Q \quad Q_0]$ is an $m \times m$ orthogonal matrix with $Q_0 \in \mathbb{R}^{m \times (m-r)}$
- columns of Q_0 are an orthonormal basis for $\text{null}(A^T)$:

$$\text{range}(Q_0) = \text{range}(A)^\perp = \text{null}(A^T)$$

this follows from

$$A^T = P[R^T \ 0] \begin{bmatrix} Q^T \\ Q_0^T \end{bmatrix}$$

and

$$A^T z = P R^T Q^T z = 0 \iff Q^T z = 0 \iff z \in \text{range}(Q_0)$$

- called *rank-revealing QR factorization* (RRQR)

Outline

- QR factorization
- QR via Gram-Schmidt
- modified Gram-Schmidt
- pivoted QR factorization
- **Householder algorithm**

Householder algorithm

- the most widely used algorithm for QR factorization (q_r in MATLAB and Julia)
- less sensitive to rounding error than Gram-Schmidt algorithm
- computes a “full” QR factorization (QR decomposition)

$$A = [Q \quad Q_0] \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad [Q \quad Q_0] \text{ orthogonal}$$

for matrix with linearly independent columns

- can be modified to compute full pivoted QR factorization
- the full Q-factor is constructed as a product of orthogonal matrices

$$[Q \quad Q_0] = H_1 H_2 \cdots H_n$$

each H_i is an $m \times m$ symmetric and orthogonal

Reflector

Reflector: the matrix

$$H = I - 2vv^T \quad \text{with } v \text{ a unit-norm vector } \|v\| = 1$$

is called an *elementary reflector*

Properties

- a reflector matrix is symmetric, and orthogonal

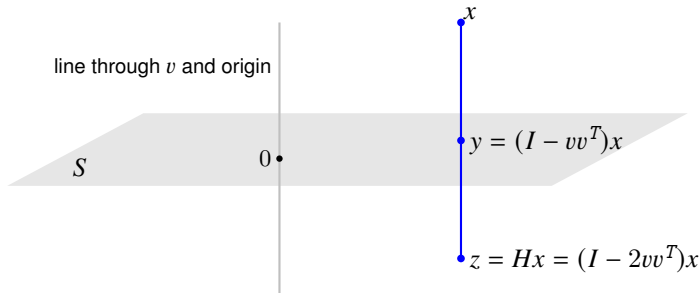
$$H^T H = (I - 2vv^T)(I - 2vv^T) = I - 4vv^T + 4vv^T vv^T = I$$

- reflection of v : $Hv = -v$
- matrix-vector product Hx can be computed efficiently as

$$Hx = x - 2(v^T x)v$$

complexity is $4p$ flops if v and x have length p

Geometrical interpretation of reflector



- $S = \{u \mid v^T u = 0\}$ is the set of vectors orthogonal to v
 - orthogonal complement of $\{\alpha v \mid \alpha \in \mathbb{R}\}$
- recall since $\|v\| = 1$, the projection of x on S is given by
- reflection of x through S is given by product with reflector:

$$z = y + (y - x) = (I - 2vv^T)x$$

Reflection to multiple of first unit vector

given nonzero p -vector $y = (y_1, y_2, \dots, y_p)$, define

$$w = \begin{bmatrix} y_1 + \text{sign}(y_1)\|y\| \\ y_2 \\ \vdots \\ y_p \end{bmatrix} = y + \text{sign}(y_1)\|y\|e_1, \quad v = \frac{1}{\|w\|}w$$

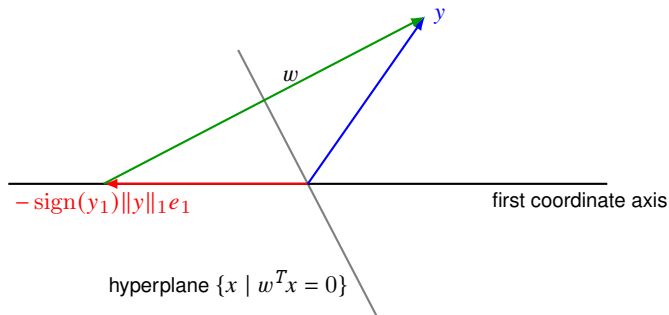
- we define $\text{sign}(0) = 1$
- vector w satisfies

$$\|w\|^2 = 2(w^T y) = 2\|y\|(\|y\| + |y_1|)$$

- reflector $H = I - 2vv^T$ maps y to multiple of $e_1 = (1, 0, \dots, 0)$:

$$Hy = y - \frac{2(w^T y)}{\|w\|^2}w = y - w = -\text{sign}(y_1)\|y\|e_1$$

Geometry



the reflection through the hyperplane $\{x \mid w^T x = 0\}$ with normal vector

$$w = y + \text{sign}(y_1)\|y\|_1 e_1$$

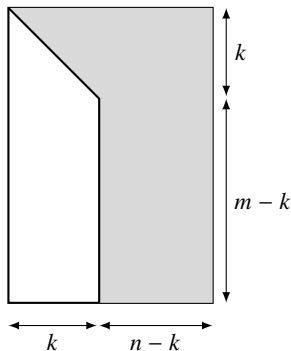
maps y to the vector $-\text{sign}(y_1)\|y\|_1 e_1$

Householder triangularization

- computes reflectors $H_1, \dots, H_n$ that reduce A to triangular form:

$$H_n H_{n-1} \cdots H_1 A = \begin{bmatrix} R \\ 0 \end{bmatrix}$$

- after step k , the matrix $H_k H_{k-1} \cdots H_1 A$ has the following structure:



(elements in positions i, j for $i > j$ and $j \leq k$ are zero)

Householder algorithm

given: $m \times n$ matrix A with linearly independent columns $a_1, \dots, a_n$

for $k = 1, 2, \dots, n$

1. define $y = A_{k:m,k}$ and compute $(m - k + 1)$ -vector v_k :

$$w = y + \text{sign}(y_1)\|y\|e_1, \quad v_k = \frac{1}{\|w\|}w$$

2. multiply $A_{k:m,k:n}$ with reflector $I - 2v_kv_k^T$:

$$A_{k:m,k:n} := A_{k:m,k:n} - 2v_k(v_k^T A_{k:m,k:n})$$

- algorithm overwrites A with $\begin{bmatrix} R \\ 0 \end{bmatrix}$
- **complexity:** $2mn^2 - \frac{2}{3}n^3$ flops (we take $2mn^2$ for the complexity)

Remarks

- step 2 is equivalent to multiplying A with $m \times m$ reflector

$$H_k = \begin{bmatrix} I & 0 \\ 0 & I - 2v_kv_k^T \end{bmatrix} = I - 2 \begin{bmatrix} 0 \\ v_k \end{bmatrix} \begin{bmatrix} 0 \\ v_k \end{bmatrix}^T$$

- algorithm returns the vectors $v_1, \dots, v_n$, with v_k of length $m - k + 1$

Q-factor

$$\begin{bmatrix} Q & Q_0 \end{bmatrix} = H_1 H_2 \cdots H_n$$

- the vectors $v_1, \dots, v_n$ are an economical representation of $\begin{bmatrix} Q & Q_0 \end{bmatrix}$
- products with $\begin{bmatrix} Q & Q_0 \end{bmatrix}$ or its transpose can be computed as

$$\begin{aligned} \begin{bmatrix} Q & Q_0 \end{bmatrix} x &= H_1 H_2 \cdots H_n x \\ \begin{bmatrix} Q & Q_0 \end{bmatrix}^T y &= H_n H_{n-1} \cdots H_1 y \end{aligned}$$

Example

$$A = \begin{bmatrix} -1 & -1 & 1 \\ 1 & 3 & 3 \\ -1 & -1 & 5 \\ 1 & 3 & 7 \end{bmatrix} = H_1 H_2 H_3 \begin{bmatrix} R \\ 0 \end{bmatrix}$$

we compute reflectors H_1, H_2, H_3 that triangularize A :

$$H_3 H_2 H_1 A = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ 0 & R_{22} & R_{23} \\ 0 & 0 & R_{33} \\ 0 & 0 & 0 \end{bmatrix}$$

First column of R

- compute reflector that maps first column of A to multiple of e_1 :

$$y = \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix}, \quad w = y - \|y\|e_1 = \begin{bmatrix} -3 \\ 1 \\ -1 \\ 1 \end{bmatrix}, \quad v_1 = \frac{1}{\|w\|}w = \frac{1}{2\sqrt{3}} \begin{bmatrix} -3 \\ 1 \\ -1 \\ 1 \end{bmatrix}$$

- overwrite A with product of $I - 2v_1v_1^T$ and A

$$A := (I - 2v_1v_1^T)A = \begin{bmatrix} 2 & 4 & 2 \\ 0 & 4/3 & 8/3 \\ 0 & 2/3 & 16/3 \\ 0 & 4/3 & 20/3 \end{bmatrix}$$

Second column of R

- compute reflector that maps $A_{2:4,2}$ to multiple of e_1 :

$$y = \begin{bmatrix} 4/3 \\ 2/3 \\ 4/3 \end{bmatrix}, \quad w = y + \|y\|e_1 = \begin{bmatrix} 10/3 \\ 2/3 \\ 4/3 \end{bmatrix}, \quad v_2 = \frac{1}{\|w\|}w = \frac{1}{\sqrt{30}} \begin{bmatrix} 5 \\ 1 \\ 2 \end{bmatrix}$$

- overwrite $A_{2:4,2:3}$ with product of $I - 2v_2v_2^T$ and $A_{2:4,2:3}$:

$$A := \begin{bmatrix} 1 & 0 \\ 0 & I - 2v_2v_2^T \end{bmatrix} A = \begin{bmatrix} 2 & 4 & 2 \\ 0 & -2 & -8 \\ 0 & 0 & 16/5 \\ 0 & 0 & 12/5 \end{bmatrix}$$

Third column of R

- compute reflector that maps $A_{3:4,3}$ to multiple of e_1 :

$$y = \begin{bmatrix} 16/5 \\ 12/5 \end{bmatrix}, \quad w = y + \|y\|e_1 = \begin{bmatrix} 36/5 \\ 12/5 \end{bmatrix}, \quad v_3 = \frac{1}{\|w\|}w = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

- overwrite $A_{3:4,3}$ with product of $I - 2v_3v_3^T$ and $A_{3:4,3}$:

$$A := \begin{bmatrix} I & 0 \\ 0 & I - 2v_3v_3^T \end{bmatrix} A = \begin{bmatrix} 2 & 4 & 2 \\ 0 & -2 & -8 \\ 0 & 0 & -4 \\ 0 & 0 & 0 \end{bmatrix}$$

Final result

$$\begin{aligned}H_3H_2H_1A &= \begin{bmatrix} I & 0 \\ 0 & I - 2v_3v_3^T \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & I - 2v_2v_2^T \end{bmatrix} (I - 2v_1v_1^T)A \\&= \begin{bmatrix} I & 0 \\ 0 & I - 2v_3v_3^T \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & I - 2v_2v_2^T \end{bmatrix} \begin{bmatrix} 2 & 4 & 2 \\ 0 & 4/3 & 8/3 \\ 0 & 2/3 & 16/3 \\ 0 & 4/3 & 20/3 \end{bmatrix} \\&= \begin{bmatrix} I & 0 \\ 0 & I - 2v_3v_3^T \end{bmatrix} \begin{bmatrix} 2 & 4 & 2 \\ 0 & -2 & -8 \\ 0 & 0 & 16/5 \\ 0 & 0 & 12/5 \end{bmatrix} \\&= \begin{bmatrix} 2 & 4 & 2 \\ 0 & -2 & -8 \\ 0 & 0 & -4 \\ 0 & 0 & 0 \end{bmatrix}\end{aligned}$$

References and further readings

- S. Boyd and L. Vandenberghe. *Introduction to Applied Linear Algebra: Vectors, Matrices, and Least Squares*. Cambridge University Press, 2018.
- L. Vandenberghe, *EE133A Lecture Notes*, University of California, Los Angeles.
- L. Vandenberghe. *EE133B Lecture Notes*, University of California, Los Angeles.
- M. T. Heath. *Scientific Computing: An Introductory Survey* (revised second edition). Society for Industrial and Applied Mathematics, 2018.
- U. M. Ascher. *A First Course on Numerical Methods*. Society for Industrial and Applied Mathematics, 2011.